

Singly Linked List Masterclass

An Interactive Step-by-Step Workbook for Beginners and Senior Engineers

Computer Science and Software Engineering Notes

Contents

How to Use This Workbook	2
1 Module 1: The Foundations	2
1.1 Intro to Linked Lists: Old and New	2
1.2 Creating a Node Class	3
1.3 Creating an Empty Linked List Wrapper	5
1.4 Finding the Length of a Linked List	6
2 Module 2: Structural Modifications	8
2.1 Insert from Head: <code>insert_head</code>	8
2.2 Traversing a Linked List: <code>__str__</code>	8
2.3 Insert from Tail: <code>append</code>	9
2.4 Inserting in the Middle: <code>insert_after</code>	10
2.5 Empty the Linked List: <code>clear</code>	11
2.6 Deleting from Head: <code>delete_head</code>	12
2.7 Deleting from Tail: <code>pop</code>	12
2.8 Delete By Value: <code>delete_by_value</code>	13
2.9 Searching and Item Retrieval: <code>search</code> and <code>__getitem__</code>	14
3 Complete Reference Implementation	16
4 Module 3: Advanced Logic and Deep-Dive Practice Problems	19
4.1 Arrays vs. Linked Lists Matrix	19
4.2 Practice Recursion II MCQs Concept	20
4.3 Problem 1: Replace Maximum Item	21
4.4 Problem 2: Sum of Odd Positions	21
4.5 Problem 3: Recursive Code Tracing <code>fun(head)</code>	22
4.6 Problem 4: Mirrored Index Pattern Identification	23
4.7 Problem 5: Complex Multi-Node Modification Trace	24
4.8 Problem 6: Linked List In-Place Reversal	26
4.9 Problem 7: Memory Compaction Architecture	27
4.10 Problem 8: Character Node String Pattern Problem	28
4.11 Problem 9: Deduplication of Sorted Linked List	29
5 Final Interview Compression Sheet	30

How to Use This Workbook

This workbook follows the supplied syllabus, reference notebook `prism-uploads/0Linked_List.ipynb`, and uploaded slide assets `prism-uploads/01_linkedList.png`, `prism-uploads/02_linkedList.png`, and `prism-uploads/03_linkedList.png`. It is intentionally chronological: master one pointer manipulation, answer the checkpoint, then proceed.

A **singly linked list** is a sequence of nodes where each node stores two things:

$$\text{node} = (\text{data}, \text{next reference})$$

The **next** reference points only forward. There is no built-in backward pointer.

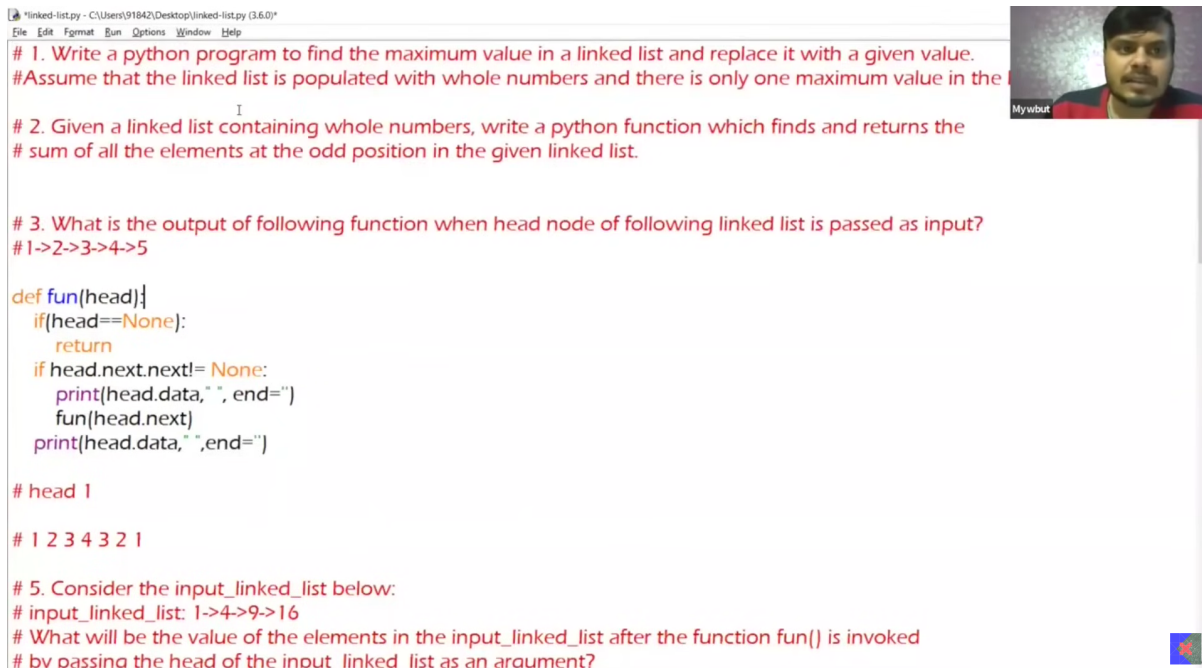
1 Module 1: The Foundations

1.1 Intro to Linked Lists: Old and New

1. The Conceptual Mental Model

Think of a **static array** like a row of reserved theater seats: seat 0, seat 1, seat 2, and so on are physically next to each other. If you know the address of seat 0 and the size of one seat, you can jump instantly to seat i .

A **singly linked list** is more like a scavenger hunt. Each clue card contains a value and the location of the next clue card. The cards do not need to live next to each other. To reach the fifth card, you must follow four links from the first card.



```
*linked-list.py - C:\Users\91842\Desktop\linked-list.py (3.6.0)*
File Edit Format Run Options Window Help

# 1. Write a python program to find the maximum value in a linked list and replace it with a given value.
# Assume that the linked list is populated with whole numbers and there is only one maximum value in the list.

# 2. Given a linked list containing whole numbers, write a python function which finds and returns the
# sum of all the elements at the odd position in the given linked list.

# 3. What is the output of following function when head node of following linked list is passed as input?
# 1->2->3->4->5

def fun(head):
    if(head==None):
        return
    if head.next.next!= None:
        print(head.data, " ", end="")
        fun(head.next)
    print(head.data, " ", end="")

# head 1
# 1 2 3 4 3 2 1

# 5. Consider the input_linked_list below:
# input_linked_list: 1->4->9->16
# What will be the value of the elements in the input_linked_list after the function fun() is invoked
# by passing the head of the input linked list as an argument?
```

Array access: $\text{base address} + i \times \text{element size}$

Linked-list access: $\text{head} \rightarrow \text{next} \rightarrow \text{next} \rightarrow \dots$

2. The Senior Engineer's Deep-Dive

Arrays are **contiguous**: neighboring logical elements usually occupy neighboring memory addresses. This helps the CPU because modern processors load memory in cache lines. When an array element is loaded into *L1* or *L2* cache, nearby elements often arrive with it, producing excellent spatial locality.

Linked-list nodes are usually allocated separately on the heap. Their logical order is maintained by references, not by physical adjacency. That means traversing a linked list often causes cache misses:

node₁ may be at address *A*, node₂ may be at address *B*, $A \not\approx B$

This is why linked lists are not automatically faster than arrays. Their strength is not random access; their strength is **cheap pointer rewiring** once you already have the insertion or deletion location.

Edge-Case Alert

A common interview trap: people say linked-list insertion is always $O(1)$. More precisely, insertion is $O(1)$ only after you already have a reference to the insertion position. Searching for that position is usually $O(n)$.

3. The Algorithm & Pseudocode

To understand a linked list:

1. Keep a reference to the first node: **head**.
2. Each node stores data and a reference to the next node.
3. The final node stores **next = None**.
4. Traversal starts at **head** and repeatedly follows **next**.
5. Stop when the current reference becomes **None**.

4. Flawless Code Implementation

```
class Node:
    """A single node in a singly linked list."""

    def __init__(self, value):
        self.data = value
        self.next = None
```

Verify Understanding Checkpoint

Short answer: Why can an array jump to index *i* in $O(1)$ time, while a singly linked list must usually walk through *i* links to reach the same logical position?

1.2 Creating a Node Class

1. The Conceptual Mental Model

A node is a box with two compartments. The first compartment stores the actual value. The second compartment stores a forwarding address to the next box.

data next

If `a.next = b`, then node `a` does not contain a copy of node `b`. It contains a reference that lets Python find object `b`.

2. The Senior Engineer's Deep-Dive

In CPython, `id(obj)` is closely related to the object's memory identity for the life of the object. When the reference notebook prints `id(a)`, `id(b)`, and `id(c)`, it is exposing that `a`, `b`, and `c` are separate objects.

Pointer-style reasoning:

- `a` is a variable referring to a `Node` object.
- `a.next` is an attribute that can refer to another `Node` object.
- `None` means "there is no next node."

Edge-Case Alert

Python references are safer than raw C/C++ pointers, but the logical bugs are the same: if you overwrite the only reference to a node before saving its successor, you can lose access to the rest of the chain.

3. The Algorithm & Pseudocode

Manual chain construction:

1. Create node `a` with value 1.
2. Create node `b` with value 2.
3. Create node `c` with value 3.
4. Set `a.next = b`.
5. Set `b.next = c`.
6. Leave `c.next = None`.

4. Flawless Code Implementation

```
class Node:
    """A node stores a value and a reference to the next node."""

    def __init__(self, value):
        self.data = value
        self.next = None

a = Node(1)
b = Node(2)
c = Node(3)

a.next = b
```

```
b.next = c

print(a.data) # 1
print(a.next.data) # 2
print(a.next.next.data) # 3
```

Verify Understanding Checkpoint

Multiple choice: After `a.next = b`, what does `a.next` store?

- A. A duplicate copy of `b.data` only.
- B. A reference to the same `Node` object that `b` refers to.
- C. The integer index of `b`.
- D. A brand-new node unrelated to `b`.

1.3 Creating an Empty Linked List Wrapper

1. The Conceptual Mental Model

The `Node` is an individual train car. The `LinkedList` wrapper is the station control office. It tracks where the train starts using `head`, and it tracks how many cars exist using `n`.

2. The Senior Engineer's Deep-Dive

A production list object should own important metadata:

- `self.head`: entry point to the chain.
- `self.n`: cached length, enabling $O(1)$ length queries.

Without `head`, all nodes become unreachable from the list. Without `n`, every call to `len(list)` would need a traversal.

3. The Algorithm & Pseudocode

Initialize an empty list:

1. Set `head` to `None` because there is no first node.
2. Set `n` to 0 because there are no nodes.

4. Flawless Code Implementation

```
class LinkedList:
    """A singly linked list wrapper tracking head and size."""

    def __init__(self):
        self.head = None
        self.n = 0
```

Verify Understanding Checkpoint

Short answer: If `self.head` is `None`, what exact structural fact is true about the linked list?

1.4 Finding the Length of a Linked List

1. The Conceptual Mental Model

There are two ways to know how many train cars are attached:

- Keep a counter updated whenever a car is added or removed.
- Walk the whole train and count each car manually.

The first is faster for repeated queries; the second is simpler but costs a traversal.

2. The Senior Engineer's Deep-Dive

If `self.n` is maintained correctly, `len(L)` is $O(1)$. If we compute length by traversal, the cost is $O(n)$.

$$T_{\text{cached length}}(n) = O(1), \quad T_{\text{manual count}}(n) = O(n)$$

The tradeoff is correctness discipline: every insert must increment `n`, and every successful delete must decrement `n`.

3. The Algorithm & Pseudocode

Cached length:

1. Return `self.n`.

Manual length:

1. Set `count = 0`.
2. Set `current = head`.
3. While `current` is not `None`, increment count and advance.
4. Return count.

4. Flawless Code Implementation

```
class LinkedList:
    def __init__(self):
        self.head = None
        self.n = 0

    def __len__(self):
        """Return cached length in O(1)."""
        return self.n

    def computed_length(self):
        """Return length by traversal in O(n), useful for validation."""
        count = 0
        current = self.head
        while current is not None:
            count += 1
            current = current.next
        return count
```

Verify Understanding Checkpoint

Multiple choice: Which bug makes cached length unreliable?

- A. Returning `self.n`.
- B. Incrementing `self.n` after insertion.
- C. Forgetting to decrement `self.n` after a successful deletion.
- D. Traversing with `current = current.next`.

2 Module 2: Structural Modifications

2.1 Insert from Head: `insert_head`

1. The Conceptual Mental Model

Inserting at the head is like placing a new train car before the current first car. The new car must first hook onto the old first car; only then should the station announce the new car as the first.

2. The Senior Engineer's Deep-Dive

Correct order matters:

1. `new_node.next = self.head`
2. `self.head = new_node`

If you reverse this order and do `self.head = new_node` first, then `self.head` no longer points to the old chain. If no other references exist, the old list becomes unreachable.

3. The Algorithm & Pseudocode

1. Create a new node.
2. Point the new node's `next` to the current head.
3. Move `head` to the new node.
4. Increment `length`.

4. Flawless Code Implementation

```
def insert_head(self, value):  
    """Insert value at the beginning in O(1)."""  
    new_node = Node(value)  
    new_node.next = self.head  
    self.head = new_node  
    self.n += 1
```

Verify Understanding Checkpoint

Short answer: Why must `new_node.next = self.head` happen before `self.head = new_node`?

2.2 Traversing a Linked List: `__str__`

1. The Conceptual Mental Model

Traversal is reading a chain of clues. Start at the first clue. Read it. Follow its next pointer. Repeat until the clue says “no next clue.”

2. The Senior Engineer's Deep-Dive

The variable `current` is not a node copy. It is a moving reference. Reassigning `current = current.next` does not mutate the list; it only changes which node the local variable points to.

3. The Algorithm & Pseudocode

1. Set `current = self.head`.
2. Initialize an empty output list.
3. While `current` is not `None`, append its data.
4. Advance `current = current.next`.
5. Join values with arrows.

4. Flawless Code Implementation

```
def __str__(self):
    """Return a readable arrow representation of the list."""
    values = []
    current = self.head
    while current is not None:
        values.append(str(current.data))
        current = current.next
    return " -> ".join(values)
```

Verify Understanding Checkpoint

Multiple choice: During traversal, what does `current = current.next` do?

- A. Deletes the current node.
- B. Copies the next node into the current node.
- C. Moves the local reference to the adjacent node.
- D. Reverses the link direction.

2.3 Insert from Tail: append

1. The Conceptual Mental Model

Appending is attaching a new train car to the end of the train. If the train is empty, the new car becomes both the first and last visible car. Without a tail pointer, we must walk to the final car.

2. The Senior Engineer's Deep-Dive

The final node is identified by:

`current.next is None`

Appending without a maintained tail pointer costs $O(n)$. With a tail pointer, it can be $O(1)$, but then deletions and edge cases require more bookkeeping.

3. The Algorithm & Pseudocode

1. Create a new node.
2. If the list is empty, set `head` to the new node.
3. Otherwise, walk until `current.next is None`.
4. Set `current.next = new_node`.
5. Increment length.

4. Flawless Code Implementation

```
def append(self, value):
    """Insert value at the end in O(n) without a tail pointer."""
    new_node = Node(value)

    if self.head is None:
        self.head = new_node
        self.n += 1
        return

    current = self.head
    while current.next is not None:
        current = current.next

    current.next = new_node
    self.n += 1
```

Verify Understanding Checkpoint

Short answer: Why does the append loop check `current.next is not None` instead of `current is not None`?

2.4 Inserting in the Middle: `insert_after`

1. The Conceptual Mental Model

To insert a new clue card after a specific card, first find the target card. Then make the new card point to the target's old next card. Finally make the target card point to the new card.

2. The Senior Engineer's Deep-Dive

The splice must preserve the suffix chain:

$$A \rightarrow B \rightarrow C$$

Insert X after B :

$$X.next = B.next, \quad B.next = X$$

If $B.next = X$ occurs first and C has no saved reference, then C and everything after it may become unreachable from the list.

3. The Algorithm & Pseudocode

1. Create the new node.
2. Traverse from head looking for `after`.
3. When found, set `new_node.next = current.next`.
4. Set `current.next = new_node`.
5. Increment length and return success.
6. If not found, return failure.

4. Flawless Code Implementation

```
def insert_after(self, after, value):
    """Insert value after the first node whose data equals after."""
    current = self.head
    while current is not None:
        if current.data == after:
            new_node = Node(value)
            new_node.next = current.next
            current.next = new_node
            self.n += 1
            return True
        current = current.next
    return False
```

Verify Understanding Checkpoint

Multiple choice: In middle insertion, which assignment protects the original suffix chain?

- A. `current = current.next`
- B. `new_node.next = current.next`
- C. `self.head = None`
- D. `self.n += 1`

2.5 Empty the Linked List: `clear`

1. The Conceptual Mental Model

If the station control office loses the address of the first train car, it can no longer reach any car in the chain. In Python, unreachable objects become eligible for garbage collection.

2. The Senior Engineer's Deep-Dive

CPython primarily uses reference counting, supported by a cyclic garbage collector. A plain singly linked list with no back-references is usually reclaimed once `self.head` is set to `None`, assuming no outside variables still reference nodes.

3. The Algorithm & Pseudocode

1. Set `self.head = None`.
2. Set `self.n = 0`.

4. Flawless Code Implementation

```
def clear(self):
    """Remove all nodes from the list."""
    self.head = None
    self.n = 0
```

Verify Understanding Checkpoint

Short answer: Why can setting only `self.head = None` make the whole chain collectible in Python?

2.6 Deleting from Head: `delete_head`

1. The Conceptual Mental Model

Deleting the head is telling the station that the second car is now the first car. The old first car is no longer reachable through the list.

2. The Senior Engineer's Deep-Dive

Empty-list handling is mandatory. For a one-node list, `self.head.next` is `None`; assigning that to `self.head` correctly produces an empty list.

3. The Algorithm & Pseudocode

1. If `head` is `None`, deletion fails.
2. Otherwise, set `head = head.next`.
3. Decrement length.

4. Flawless Code Implementation

```
def delete_head(self):
    """Delete the first node, if present."""
    if self.head is None:
        return False

    self.head = self.head.next
    self.n -= 1
    return True
```

Verify Understanding Checkpoint

Multiple choice: What happens when `delete_head` runs on a one-node list?

- A. `head` becomes `None`.
- B. The node points to itself.
- C. The length increases.
- D. Python raises an error automatically.

2.7 Deleting from Tail: `pop`

1. The Conceptual Mental Model

To remove the final train car, you need access to the car before it. That second-to-last car must have its `next` reference changed to `None`.

2. The Senior Engineer's Deep-Dive

The key condition is:

`current.next.next is None`

This means `current.next` is the tail, so `current` is the second-to-last node.

3. The Algorithm & Pseudocode

1. If empty, fail.
2. If one node, set `head = None`.
3. Otherwise, walk until `current.next.next` is `None`.
4. Set `current.next = None`.
5. Decrement length.

4. Flawless Code Implementation

```
def pop(self):
    """Delete the last node and return whether deletion occurred."""
    if self.head is None:
        return False

    if self.head.next is None:
        self.head = None
        self.n -= 1
        return True

    current = self.head
    while current.next.next is not None:
        current = current.next

    current.next = None
    self.n -= 1
    return True
```

Verify Understanding Checkpoint

Short answer: Why is `current.next.next` is `None` the correct signal that `current` is second-to-last?

2.8 Delete By Value: `delete_by_value`

1. The Conceptual Mental Model

Deleting by value is like removing a specific train car by label. If it is the first car, move the head. If it is in the middle, make the previous car skip it.

2. The Senior Engineer's Deep-Dive

We need the predecessor because singly linked lists cannot move backward. To delete `target`, perform:

```
previous.next = target.next
```

Head deletion is special because there is no previous node.

3. The Algorithm & Pseudocode

1. If empty, fail.

2. If head has the value, delete head.
3. Otherwise keep `previous` and `current` references.
4. Traverse until the value is found.
5. Rewire `previous.next` to skip `current`.
6. Decrement length.

4. Flawless Code Implementation

```
def delete_by_value(self, value):
    """Delete the first node whose data equals value."""
    if self.head is None:
        return False

    if self.head.data == value:
        return self.delete_head()

    previous = self.head
    current = self.head.next

    while current is not None:
        if current.data == value:
            previous.next = current.next
            self.n -= 1
            return True
        previous = current
        current = current.next

    return False
```

Verify Understanding Checkpoint

Multiple choice: Why do we track `previous` in delete-by-value?

- A. To print the list backward.
- B. To rewire `previous.next` around the deleted node.
- C. To avoid using `None`.
- D. To make random access $O(1)$.

2.9 Searching and Item Retrieval: `search` and `__getitem__`

1. The Conceptual Mental Model

Searching by value asks, “Where is the first card with this label?” Index retrieval asks, “What value is stored on the card at step i from the head?”

2. The Senior Engineer’s Deep-Dive

Python bracket syntax `L[i]` is implemented by `__getitem__`. A linked list can support the syntax, but not array-like speed. Accessing `L[i]` costs $O(i)$, which is $O(n)$ in the worst case.

3. The Algorithm & Pseudocode

Search:

1. Set index to 0 and current to head.
2. Walk through nodes.
3. If data matches, return index.
4. If traversal ends, return -1.

Get item:

1. Reject negative or out-of-range indexes.
2. Walk forward exactly `index` steps.
3. Return `current.data`.

4. Flawless Code Implementation

```
def search(self, value):
    """Return the index of the first matching value, or -1."""
    index = 0
    current = self.head
    while current is not None:
        if current.data == value:
            return index
        index += 1
        current = current.next
    return -1

def __getitem__(self, index):
    """Return the value at index using bracket syntax."""
    if index < 0 or index >= self.n:
        raise IndexError("linked list index out of range")

    current = self.head
    for _ in range(index):
        current = current.next
    return current.data
```

Verify Understanding Checkpoint

Short answer: Why does `L[500]` on a linked list not have the same performance profile as `array[500]`?

3 Complete Reference Implementation

```
class Node:
    """A single node in a singly linked list."""

    def __init__(self, value):
        self.data = value
        self.next = None

class LinkedList:
    """Production-quality singly linked list matching the syllabus."""

    def __init__(self):
        self.head = None
        self.n = 0

    def __len__(self):
        return self.n

    def __str__(self):
        values = []
        current = self.head
        while current is not None:
            values.append(str(current.data))
            current = current.next
        return " -> ".join(values)

    def insert_head(self, value):
        new_node = Node(value)
        new_node.next = self.head
        self.head = new_node
        self.n += 1

    def append(self, value):
        new_node = Node(value)
        if self.head is None:
            self.head = new_node
            self.n += 1
            return

        current = self.head
        while current.next is not None:
            current = current.next
        current.next = new_node
        self.n += 1

    def insert_after(self, after, value):
        current = self.head
        while current is not None:
            if current.data == after:
                new_node = Node(value)
                new_node.next = current.next
                current.next = new_node
                self.n += 1
                return True
            current = current.next
```

```

        current = current.next
    return False

def clear(self):
    self.head = None
    self.n = 0

def delete_head(self):
    if self.head is None:
        return False
    self.head = self.head.next
    self.n -= 1
    return True

def pop(self):
    if self.head is None:
        return False
    if self.head.next is None:
        self.head = None
        self.n -= 1
        return True

    current = self.head
    while current.next.next is not None:
        current = current.next
    current.next = None
    self.n -= 1
    return True

def delete_by_value(self, value):
    if self.head is None:
        return False
    if self.head.data == value:
        return self.delete_head()

    previous = self.head
    current = self.head.next
    while current is not None:
        if current.data == value:
            previous.next = current.next
            self.n -= 1
            return True
        previous = current
        current = current.next
    return False

def search(self, value):
    index = 0
    current = self.head
    while current is not None:
        if current.data == value:
            return index
        index += 1
        current = current.next
    return -1

```

```
def __getitem__(self, index):
    if index < 0 or index >= self.n:
        raise IndexError("linked list index out of range")
    current = self.head
    for _ in range(index):
        current = current.next
    return current.data
```

Verify Understanding Checkpoint

Short answer: Which methods must update `self.n`, and why is that bookkeeping essential for `__len__` and `__getitem__`?

4 Module 3: Advanced Logic and Deep-Dive Practice Problems

4.1 Arrays vs. Linked Lists Matrix

1. The Conceptual Mental Model

Arrays are highways with numbered exits. Linked lists are trails with signs pointing to the next landmark. Arrays excel when you know the exact index. Linked lists excel when you already hold the node where a splice should occur.

2. The Senior Engineer's Deep-Dive

The following table separates theoretical complexity from architecture-level performance. Big-O ignores cache behavior, allocation overhead, and branch prediction, but real systems do not.

3. The Algorithm & Pseudocode

Choose arrays when random access and cache locality dominate. Choose linked lists when frequent local splicing dominates and direct node references are available.

4. Comparison Matrix

Operation	Array	Singly Linked List	Key Engineering Note
Random access by index	$O(1)$	$O(n)$	Arrays use address arithmetic; lists traverse.
Search by value	$O(n)$	$O(n)$	Both may scan all elements.
Insert at head	$O(n)$	$O(1)$	Arrays shift; lists rewire head.
Insert at tail	Amortized $O(1)$ dynamic array	$O(n)$ without tail, $O(1)$ with tail	Tail metadata changes the list cost.
Insert after known node	$O(n)$ shifting after position	$O(1)$	Linked list wins if node reference is already known.
Delete head	$O(n)$	$O(1)$	Arrays shift left; lists move head.
Delete tail	$O(1)$ dynamic array	$O(n)$ singly list without previous pointer	Singly lists need second-to-last node.
Memory overhead	Low per element	Higher per node	Each node stores data plus reference.
Cache locality	Strong	Weak	Arrays are contiguous; nodes may be scattered.

Verify Understanding Checkpoint

Multiple choice: When is linked-list insertion truly $O(1)$?

- A. Whenever the list is nonempty.
- B. Whenever the value is small.
- C. When you already have the relevant node reference or are inserting at head.
- D. When Python garbage collection is enabled.

4.2 Practice Recursion II MCQs Concept

1. The Conceptual Mental Model

Recursive linked-list processing is a stack of unfinished promises. On the forward pass, each call says, “I will finish after the smaller suffix finishes.” During unwinding, calls resume in reverse order.

2. The Senior Engineer’s Deep-Dive

Every recursive call has its own stack frame. A frame stores local variables, the current **head** reference, and the return location. For a list of length n , a simple recursive traversal uses $O(n)$ call-stack space.

3. The Algorithm & Pseudocode

Forward-order print:

1. Print current node.
2. Recurse on next node.

Reverse-order print:

1. Recurse on next node.
2. Print current node during unwinding.

4. Flawless Code Implementation

```
def print_forward(head):
    if head is None:
        return
    print(head.data)
    print_forward(head.next)

def print_reverse(head):
    if head is None:
        return
    print_reverse(head.next)
    print(head.data)
```

Verify Understanding Checkpoint

Short answer: In recursive reverse printing, why does the tail print before the head even though the function starts at the head?

4.3 Problem 1: Replace Maximum Item

1. The Conceptual Mental Model

Walk through the train once while carrying a sticky note saying, “best node seen so far.” At the end, mutate that node’s data.

2. The Senior Engineer’s Deep-Dive

Do not make two passes unless required. Store a reference to the maximum node, not merely the maximum value, because the final operation mutates the node object.

3. The Algorithm & Pseudocode

1. If the list is empty, fail.
2. Set `max_node = head`.
3. Traverse from `head.next`.
4. If `current.data > max_node.data`, update `max_node`.
5. After traversal, set `max_node.data = value`.

4. Flawless Code Implementation

```
def replace_max(self, value):
    """Replace the maximum data item with value in one traversal."""
    if self.head is None:
        return False

    max_node = self.head
    current = self.head.next
    while current is not None:
        if current.data > max_node.data:
            max_node = current
        current = current.next

    max_node.data = value
    return True
```

Verify Understanding Checkpoint

Multiple choice: Why store `max_node` rather than only `max_value`?

- A. Because we need the node reference to mutate its `data`.
- B. Because integers cannot be compared.
- C. Because traversal requires two pointers always.
- D. Because `self.head` must be deleted.

4.4 Problem 2: Sum of Odd Positions

1. The Conceptual Mental Model

Imagine numbering nodes as you walk: head is index 0, next is index 1, then 2, then 3. Add only the values on odd-numbered cards.

2. The Senior Engineer's Deep-Dive

Odd position means odd index, not odd value. Node with value 10 at index 1 contributes; node with value 9 at index 2 does not.

3. The Algorithm & Pseudocode

1. Initialize `index = 0` and `total = 0`.
2. Traverse nodes.
3. If `index % 2 == 1`, add `current.data`.
4. Increment index and advance.
5. Return total.

4. Flawless Code Implementation

```
def sum_odd_positions(self):
    """Sum values at odd indexes, with head at index 0."""
    index = 0
    total = 0
    current = self.head

    while current is not None:
        if index % 2 == 1:
            total += current.data
        index += 1
        current = current.next

    return total
```

Verify Understanding Checkpoint

Short answer: In the list 5 -> 100 -> 7 -> 20, what is the odd-position sum and why?

4.5 Problem 3: Recursive Code Tracing `fun(head)`

1. The Conceptual Mental Model

This function prints on the way down and again on the way back up, but only for nodes that have at least two successors ahead of them.

2. The Senior Engineer's Deep-Dive

The condition `head.next.next != None` is dangerous if `head.next` is `None`. For the input 1->2->3->4->5, the function avoids the null problem because it stops recursing at node 3. In production, the condition should guard `head.next` first.

3. The Algorithm & Stack Trace

Given:

```
def fun(head):
    if head == None: return
    if head.next.next != None:
        print(head.data, end=" ")
        fun(head.next)
    print(head.data, end=" ")
```

For 1->2->3->4->5:

1. At node 1, 1.next.next is node 3, so print 1, recurse to 2.
2. At node 2, 2.next.next is node 4, so print 2, recurse to 3.
3. At node 3, 3.next.next is node 5, so print 3, recurse to 4.
4. At node 4, 4.next.next is None, so do not enter the block and return.
5. Unwind to node 3, print 3.
6. Unwind to node 2, print 2.
7. Unwind to node 1, print 1.

Output:

1 2 3 3 2 1

4. Safer Code Implementation

```
def fun_safe(head):
    if head is None:
        return
    if head.next is not None and head.next.next is not None:
        print(head.data, end=" ")
        fun_safe(head.next)
    print(head.data, end=" ")
```

Verify Understanding Checkpoint

Multiple choice: Why does node 4 not print in the original trace?

- A. Its data is even.
- B. head.next.next is None at node 4.
- C. Recursive calls skip every fourth node.
- D. The function deletes node 4.

4.6 Problem 4: Mirrored Index Pattern Identification

1. The Conceptual Mental Model

A mirrored output such as 1 2 3 4 3 2 1 is a mountain path: climb from the head toward a center, then walk back down during recursion unwinding.

2. The Senior Engineer's Deep-Dive

The forward phase emits increasing positions. The unwinding phase emits previous positions in reverse. To avoid printing the center twice, print after recursion only when the current node is not the center

or control the base case carefully.

3. The Algorithm & Pseudocode

For a target depth k :

1. Print current node on entry.
2. If current index equals k , stop descending.
3. Recurse to next node.
4. Print current node on return.

4. Flawless Code Implementation

```
def print_mirror_prefix(head, depth):
    """Print nodes from index 0..depth and back to 0."""
    def helper(node, index):
        if node is None:
            return

        print(node.data, end=" ")
        if index == depth:
            return

        helper(node.next, index + 1)
        print(node.data, end=" ")

    helper(head, 0)
```

For data 1->2->3->4->5 and depth = 3, output is 1 2 3 4 3 2 1.

Verify Understanding Checkpoint

Short answer: Which phase of recursion produces the second 3 2 1 in 1 2 3 4 3 2 1: forward calls or unwinding?

4.7 Problem 5: Complex Multi-Node Modification Trace

1. The Conceptual Mental Model

The loop walks adjacent pairs and repeatedly adds the next node's current value into the current node. Since data is mutated as the algorithm runs, later calculations may use already-modified values.

2. The Senior Engineer's Deep-Dive

The function is:

```
def fun(head):
    next_node = head.next
    while head != None and next_node != None:
        head.data = head.data + next_node.data
        head = head.next
        next_node = head.next
    if next_node != None:
```

```
head.data = head.data + next_node.data
```

Input:

$1 \rightarrow 4 \rightarrow 9 \rightarrow 16$

Trace:

1. Start: `head=1`, `next_node=4`.
2. Set first node: $1 + 4 = 5$. List becomes $5 \rightarrow 4 \rightarrow 9 \rightarrow 16$.
3. Move head to second node 4; next is 9.
4. Since next exists, set second node: $4 + 9 = 13$. List becomes $5 \rightarrow 13 \rightarrow 9 \rightarrow 16$.
5. Loop condition with `head=13`, `next_node=9`.
6. Set second node again: $13 + 9 = 22$. List becomes $5 \rightarrow 22 \rightarrow 9 \rightarrow 16$.
7. Move head to third node 9; next is 16.
8. Since next exists, set third node: $9 + 16 = 25$. List becomes $5 \rightarrow 22 \rightarrow 25 \rightarrow 16$.
9. Loop condition with `head=25`, `next_node=16`.
10. Set third node again: $25 + 16 = 41$. List becomes $5 \rightarrow 22 \rightarrow 41 \rightarrow 16$.
11. Move head to fourth node; `next_node=None`; skip inner if and exit loop.

Final state:

$5 \rightarrow 22 \rightarrow 41 \rightarrow 16$

3. The Algorithm & Pseudocode

The algorithm mutates node i using node $i + 1$, then mutates node $i + 1$ using node $i + 2$ inside the same loop. This causes overlap.

4. Flawless Code Implementation

```
def multi_node_trace_mutation(head):  
    if head is None:  
        return  
  
    next_node = head.next  
    while head is not None and next_node is not None:  
        head.data = head.data + next_node.data  
        head = head.next  
        next_node = head.next  
        if next_node is not None:  
            head.data = head.data + next_node.data
```

Verify Understanding Checkpoint

Multiple choice: Why does the second node become 22 rather than 13?

- A. It is updated once by $4 + 9$ and then again by $13 + 9$.
- B. The first node copies into it.
- C. Python automatically doubles middle nodes.
- D. The loop skips the second node.

4.8 Problem 6: Linked List In-Place Reversal

1. The Conceptual Mental Model

Reversal is flipping every arrow. To avoid losing the rest of the chain, carry three hands: one on the previous node, one on the current node, and one saving the next node.

2. The Senior Engineer's Deep-Dive

The three-pointer invariant:

- `prev`: reversed prefix.
- `curr`: node currently being reversed.
- `next_node`: unreversed suffix saved before breaking the link.

Auxiliary space is $O(1)$ because the algorithm uses a constant number of references. Runtime is $O(n)$ because each node is visited once.

3. The Algorithm & Pseudocode

1. Set `prev = None`.
2. Set `curr = head`.
3. While `curr` exists, save `next_node = curr.next`.
4. Reverse current arrow: `curr.next = prev`.
5. Advance `prev = curr` and `curr = next_node`.
6. Set `head = prev`.

4. Flawless Code Implementation

```
def reverse(self):
    """Reverse the list in-place using O(1) auxiliary space."""
    prev = None
    curr = self.head

    while curr is not None:
        next_node = curr.next
        curr.next = prev
        prev = curr
        curr = next_node

    self.head = prev
```

Verify Understanding Checkpoint

Short answer: What goes wrong if you execute `curr.next = prev` before saving `next_node = curr.next`?

4.9 Problem 7: Memory Compaction Architecture

1. The Conceptual Mental Model

Heap memory can become like a parking lot with many small empty spaces between cars. There may be enough total free space, but not enough contiguous space for a large object.

2. The Senior Engineer's Deep-Dive

Fragmentation matters because linked-list nodes are individually allocated. A runtime with moving garbage collection may compact live objects by relocating them and updating references. A runtime with non-moving allocation may instead use free lists, arenas, pools, or segregated size classes.

In lower-level languages, raw pointers make compaction difficult because every pointer must be found and updated safely. Managed runtimes can track references precisely, so object relocation is more feasible.

3. The Algorithm & Pseudocode

Conceptual moving compactor:

1. Identify live objects reachable from roots.
2. Compute new compact addresses.
3. Move live objects into compact regions.
4. Rewrite all references to point to new locations.
5. Mark old fragmented space as free.

4. Engineering Summary

Linked lists reduce the need for contiguous logical storage, but they do not eliminate heap fragmentation costs. They trade contiguous layout for pointer flexibility.

Verify Understanding Checkpoint

Multiple choice: Why is memory compaction easier in managed runtimes than with arbitrary raw pointers?

- A. Managed runtimes can identify and update object references.
- B. Managed runtimes never allocate memory.
- C. Raw pointers are always stored in arrays.
- D. Linked lists cannot fragment memory.

4.10 Problem 8: Character Node String Pattern Problem

1. The Conceptual Mental Model

The character list is a stream. Delimiters * and / act like word separators. A delimiter pair means: output one space and capitalize the next word.

2. The Senior Engineer's Deep-Dive

The tricky part is consuming consecutive delimiters correctly. Any single delimiter emits one space. If the delimiter is followed immediately by another delimiter, the pair still emits only one space, and it sets `capitalize_next = True`. This covers **, //, */ , and /*.

3. The Algorithm & Pseudocode

1. Traverse the character nodes.
2. If current char is not a delimiter, output it, capitalizing if requested.
3. If current char is a delimiter, append one space unless the output already ends in a space.
4. If the next char is also a delimiter, consume it and set capitalization for the next real character.
5. Continue until the list ends.

4. Flawless Code Implementation

```
def decode_character_list(head):
    """Decode delimiter rules from a character linked list."""
    delimiters = {"*", "/" }
    output = []
    capitalize_next = False
    current = head

    while current is not None:
        char = current.data

        if char in delimiters:
            if output and output[-1] != " ":
                output.append(" ")

            if current.next is not None and current.next.data in delimiters:
                capitalize_next = True
                current = current.next.next
            else:
                current = current.next
            continue

        if capitalize_next:
            output.append(char.upper())
            capitalize_next = False
        else:
            output.append(char)

        current = current.next

    return "".join(output).strip()
```

For the supplied input, the output is:

An Apple a day Keeps A Doctor Away

Verify Understanding Checkpoint

Short answer: In the sequence `k->e->e->p->s->/->*->a`, why does the `a` become `A` while only one space is emitted?

4.11 Problem 9: Deduplication of Sorted Linked List

1. The Conceptual Mental Model

Because the list is sorted, duplicates appear in consecutive blocks. You do not need a hash set. You only need to compare each node with its immediate neighbor.

2. The Senior Engineer's Deep-Dive

This is an in-place compression problem. If `current.data == current.next.data`, bypass `current.next`. Do not advance `current` in that case, because there may be more duplicates immediately after.

3. The Algorithm & Pseudocode

1. Set `current = head`.
2. While `current` and `current.next` exist:
3. If values match, set `current.next = current.next.next` and decrement length.
4. Otherwise, advance `current = current.next`.

4. Flawless Code Implementation

```
def remove_duplicates_sorted(self):
    """Remove consecutive duplicates from a sorted linked list."""
    current = self.head

    while current is not None and current.next is not None:
        if current.data == current.next.data:
            current.next = current.next.next
            self.n -= 1
        else:
            current = current.next
```

Example:

1 -> 1 -> 2 -> 3 -> 3 -> 3 -> 4 $\implies$ 1 -> 2 -> 3 -> 4

Verify Understanding Checkpoint

Multiple choice: Why should `current` stay on the same node after deleting a duplicate neighbor?

- A. There may be another duplicate immediately after the removed node.
- B. The list becomes circular.
- C. Python requires two deletions per duplicate.
- D. Sorted lists cannot contain triples.

5 Final Interview Compression Sheet

- **Head insertion:** save old head in `new_node.next`, then move head.
- **Tail deletion:** find the second-to-last node, then set its `next` to `None`.
- **Middle insertion:** connect new node to suffix first, then predecessor to new node.
- **Middle deletion:** predecessor skips target using `previous.next = current.next`.
- **Traversal:** local pointer movement does not mutate the list.
- **Reversal:** always save `next_node` before flipping `curr.next`.
- **Performance:** arrays win random access and cache locality; lists win local pointer splicing.

Verify Understanding Checkpoint

Capstone: Given $A \rightarrow B \rightarrow C$, write the exact two assignments needed to insert X after B without losing C . Then write the exact assignment needed to delete B if you have a reference to A .